Technical Program & Product Case Study

VitalSync

Alejandra Badia
Technical Program Manager

End-to-End system design, prioritization, and implementation across data, backend, and UI layers

1. Project Overview

1.1 Project Title

VitalSync – Observability & Data Integration Monitoring Platform

1.2 Project Goal

Design a prototype observability platform to improve visibility into system health, data flows, and interoperability across multi-layer healthcare systems, including clinical, integration, reporting, and research domains.

The platform simulates these system layers to reflect real-world healthcare interoperability environments and enable monitoring across operational and integration workflows.

1.3 My Role: Product Manager & Developer

1.3.1 Product Leadership

- Defined system scope and key product decisions (observability model, architecture-first approach)
- Prioritized features based on system dependencies and user value
- Structured delivery across iterations (architecture → integration → observability)

1.3.2 Execution

- Implemented core system logic, data models, and dashboards
- Resolved data consistency and system behavior challenges

1.4 Methodology

Agile iterative, product-driven development, including the following:

- Feature-based backlog management

- Iterative delivery (MVP → enhancements)
- Continuous testing and system validation
- Focus on system behavior and data flow integrity

1.5 System Relevance

Modern distributed systems depend on reliable data pipelines, interoperability, and visibility into system behavior.

VitalSync simulates these conditions by modeling how data flows across multiple system layers, enabling monitoring, traceability, and consistency across integrated environments.

1.6 Data Model Design

- Designed a relational schema with separation between:
  - transactional data (patients)
  - event logs (system_logs, fhir_logs)
  - reporting layer (patient_sync_summary)
- Implemented **denormalized reporting table** to support a fast reporting layer
- Structured system to support:
  - event tracking (HL7-style logs)
  - auditability (FHIR payload storage)
  - performance monitoring (response time, retries)

2. Backlog Management

2.1 The Problem

Designing a system that monitors distributed data flows requires balancing multiple components (data sources, logs, dashboards), creating a risk of unclear scope and fragmented system behavior.

2.2 Key Product Decision

Treat the platform as a multi-layer system simulation representing real-world data infrastructure, rather than a standalone dashboard.

2.3 Key Product Tradeoffs

- Prioritized architecture and data modeling over advanced UI features
- Simulated system behavior instead of integrating real systems to maintain scope
- Focused on core observability (health, logs, flows) over advanced analytics

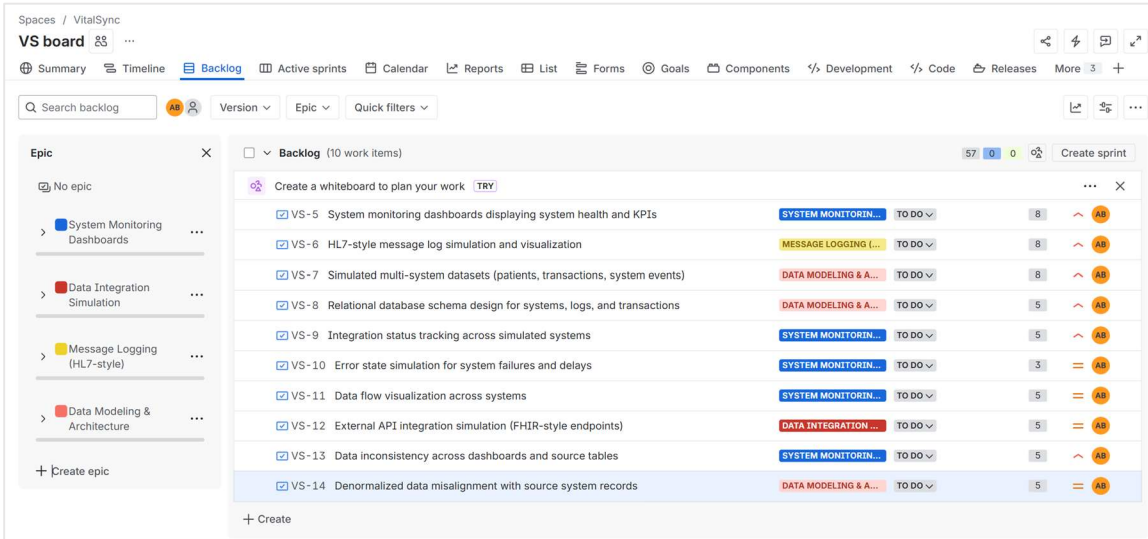
2.4 Approach / Decision

Created a structured backlog organized by epics and user stories with acceptance criteria. Prioritized c**ore observability features first (MVP)** and deferred advanced enhancements. The four epics consisted of the following:

Jira ID	EPIC	Description
VS-1	System Monitoring Dashboards	Design and development of dashboards that provide visibility into system health, integration status, and operational performance across simulated systems. This includes defining key metrics, status indicators, and visual representations that enable users to monitor system behavior and identify issues.
VS-2	Data Integration Simulation	Simulation of data exchange between multiple systems to replicate real-world integration workflows. This includes generating synthetic data, modeling system interactions, and representing how data flows across distributed environments.
VS-3	Message Logging (HL7-style)	Implementation of structured message logging to simulate healthcare-style data exchanges using HL7-like formats. This includes capturing, storing, and displaying message events to provide traceability of system interactions and support observability.
VS-4	Data Modeling & Architecture	Design of the platform's data architecture, including relational schema definition, data relationships, and handling of normalized and denormalized structures. Ensures data integrity, consistency across system components, and scalability to support simulated multi-system environments.

Table 1 - EPICS: High-level product epics defining the core components of the VitalSync platform, including system monitoring, data integration, message logging, and data architecture. These epics structured the backlog to align development with system-level design and observability goals.

2.5 Jira Backlog Screenshot



Screenshot 1 - Project Backlog: Structured product backlog showing features and data-related issues organized by epics, priorities, and story points. Items were categorized into system monitoring, data integration, message logging, and architecture to ensure clear scope definition and alignment with platform design. Data integrity issues that were identified during the development of the platform were tracked alongside features to reflect the real-world system refinement that took place during development.

2.6 Backlog Table

ID	Jira ID	Type	Summary	Epic	Story Points	Priority	Acceptance Criteria
F01	VS-5	Story	System monitoring dashboard displaying system health and KPIs	System Monitoring Dashboards	8	High	Dashboards display system status, KPIs, and integration health indicators
F02	VS-6	Story	HL7-style message log simulation and visualization	Message Logging (HL7-style)	8	High	Message logs capture events with timestamps, statuses, and are viewable in UI
F03	VS-7	Story	Simulated multi-system datasets (patients, transactions, system events)	Data Modeling & Architecture	8	High	Database populated with structured datasets representing multiple systems and events

ID	Jira ID	Type	Summary	Epic	Story Points	Priority	Acceptance Criteria
F04	VS-8	Story	Relational database schema design for systems, logs, and transactions	Data Modeling & Architecture	5	High	Schema supports relationships between systems, logs, and operational data
F05	VS-9	Story	Integration status tracking across simulated systems	System Monitoring Dashboards	5	High	Dashboards reflect real-time status of integrations (success, error, pending)
F06	VS-10	Story	Error state simulation for system failures and delays	System Monitoring Dashboards	3	Medium	System displays simulated failure states and reflects them in monitoring dashboard
F07	VS-11	Story	Data flow visualization across systems	System Monitoring Dashboards	5	Medium	UI represents relationships and flow between systems and data entities
F08	VS-12	Story	External API integration simulation (FHIR-style endpoints)	Data Integration Simulation	5	Medium	System retrieves and processes data from simulated external API endpoint
D01	VS-13	Issue	Data inconsistency across dashboards and source tables	System Monitoring Dashboards	5	High	Dashboards, logs, and data tables reflect aligned and consistent system information
D02	VS-14	Issue	Denormalized data misalignment with source system records	Data Modeling & Architecture	5	Medium	Aggregated / derived data correctly reflects underlying normalized data

Table 2 - Backlog: Structured product backlog capturing features and data-related issues across epics, priorities, and story points. Reflects both planned functionality and data integrity challenges identified during iterative system development. Note: Issue IDs (VS-#) are automatically generated by Jira and are not sequential by item type.

3. Sprint 1 – Core Platform & Architecture

3.1 Sprint Goal

Sprint 1 focused on establishing a stable system foundation to support future integration and observability features, including:

- Data model
- System structure
- Basic dashboards

3.2 Initial Sprint State

3.2.1 Problem

The system required a solid architecture before building UI features.

3.2.2 Decision

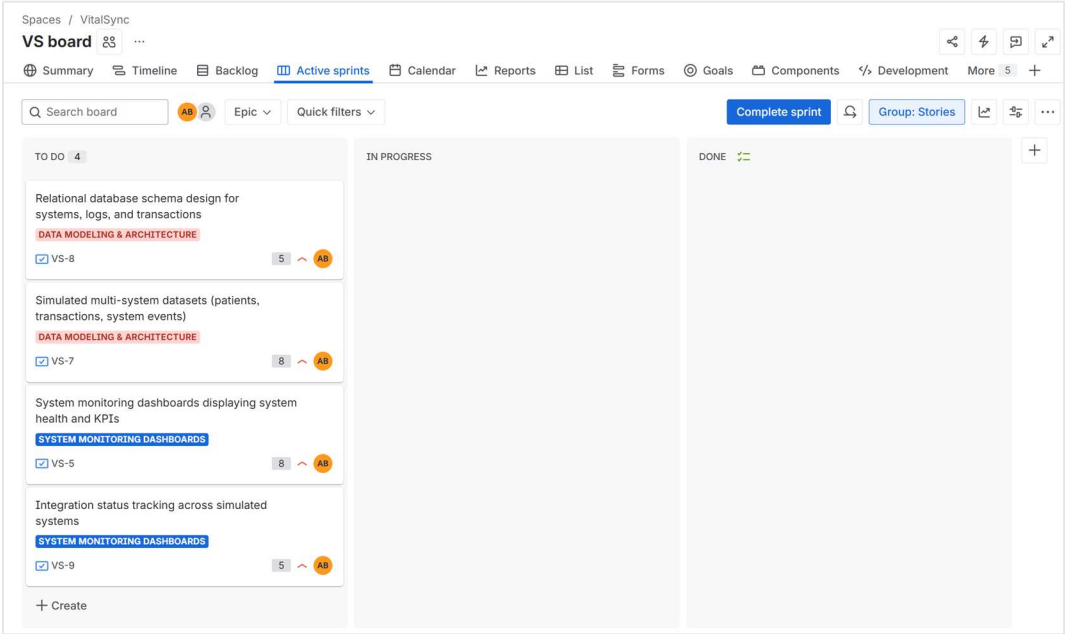
Sprint 1 focus areas:

- Database schema design
- Core data entities
- Initial dashboard structure

Deferred to subsequent sprint:

- Advanced UI features
- Complex integrations

3.2.3 Sprint 1 To-Do Screenshot



Screenshot 2 - Sprint 1 Initial State: Sprint 1 backlog focused on establishing the platform foundation, including database schema design, system simulation, and initial monitoring capabilities. Scope was intentionally limited to core architectural components and basic dashboards, while more advanced integrations and observability features were deferred to later iterations.

3.3 Work in Progress

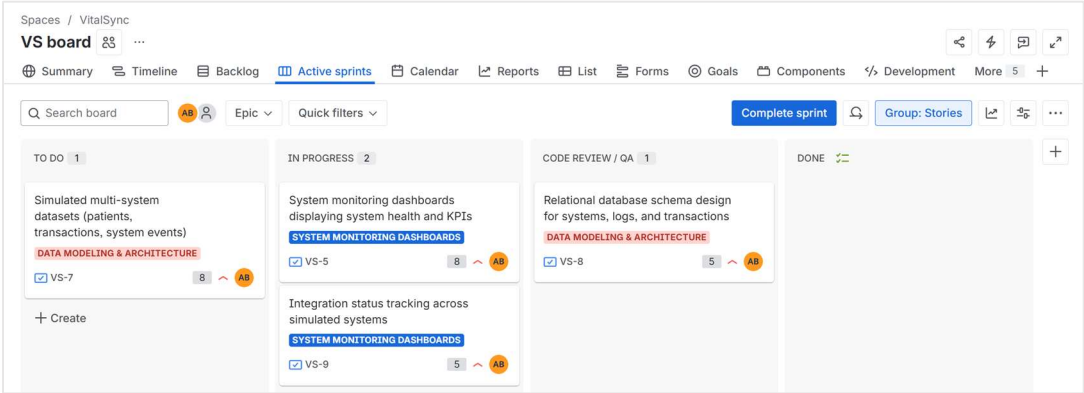
3.3.1 Challenges and Approach

Establishing the system foundation required designing scalable data relationships between systems, transactions, and logs while ensuring consistency across simulated data.

To address these dependencies:

- Designed a structured relational schema (systems, logs/messages, status tracking)
- Established a clear data model to support future integration and observability layers
- Built initial dashboards to validate system health and core metrics

3.3.2 Sprint 1 In-Progress Screenshot



Screenshot 3 - Sprint 1 In-Progress State: Sprint 1 in progress, showing active development of core system components. Work focused on defining data relationships, building the initial schema, and implementing foundational dashboard elements. Dependencies between data modeling and dashboard functionality were managed to ensure consistent system behavior.

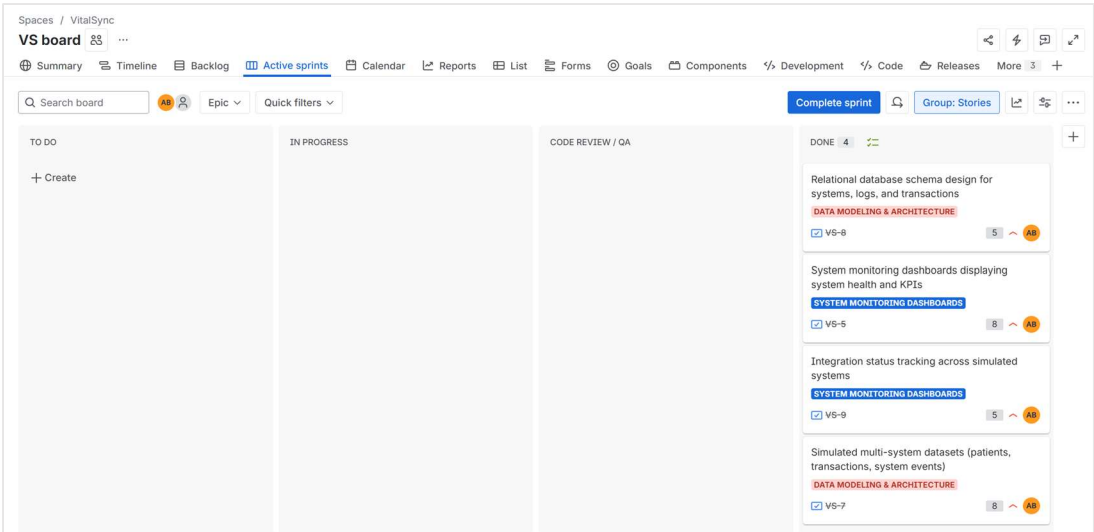
3.4 Sprint Completion

3.4.1 Outcome

Established:

- Stable data model
- Working dashboard framework
- Initial system simulation

3.4.2 Sprint 1 Completion Screenshot



Screenshot 4 - Sprint 1 Completion: Sprint 1 completed with delivery of a stable data model, initial system simulation, and working dashboard framework. The platform achieved a functional baseline, enabling further development of integration and observability features in subsequent iterations.

4. Sprint 2 – Integration Simulation & Observability

4.1 Sprint Goal

Sprint 2 focused on translating the foundational architecture into realistic system behavior and observable outputs, including:

- Data flows
- Integration status
- Message logs

This allowed connecting system layers and enabling observability across clinical data, integration workflows, and reporting outputs.

4.2 Initial Sprint State

4.2.1 Problem

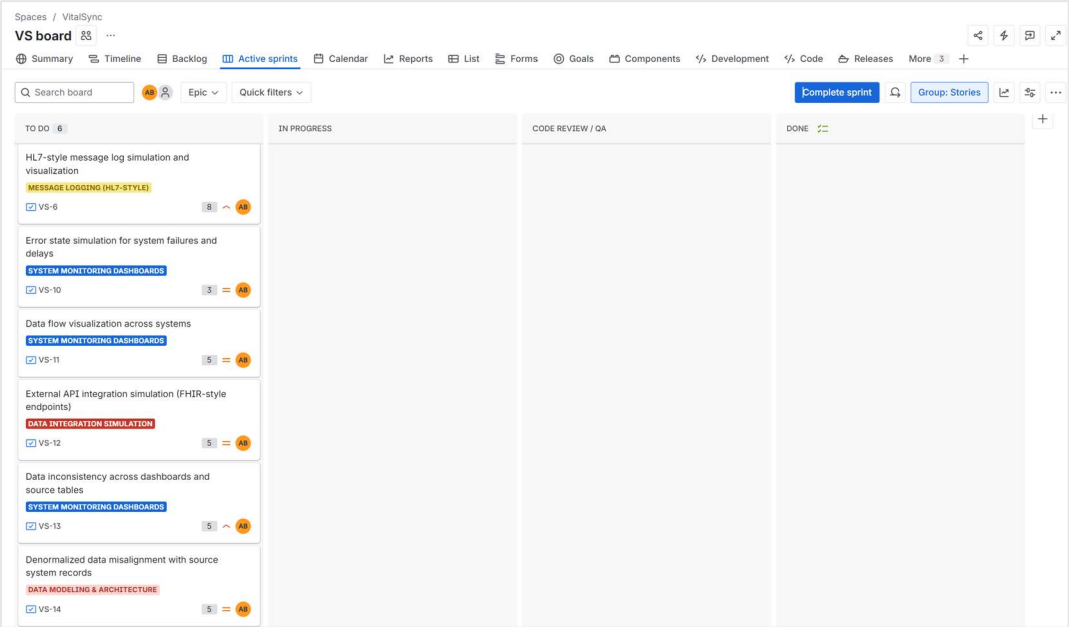
Simulating multiple interacting systems introduced complexity in maintaining data consistency, synchronization, and realistic system behavior.

4.2.2 Decision (Scope Management)

Focused on:

- HL7-style message logging and event simulation
- integration state tracking
- dashboard visualization

4.2.3 Sprint 2 To-Do Screenshot



Screenshot 5 - Sprint 2 Initial State: Sprint 2 backlog focused on enhancing system observability and simulating real-world system behavior. Scope included message logging, integration monitoring, and visualization, along with resolving data consistency and modeling issues identified during system integration.

4.3 Work in Progress

4.3.1 Key System Challenges & Resolutions

The following challenges emerged while simulating real-world system interactions and were **addressed through targeted design decisions:**

No.	Challenge	Resolution

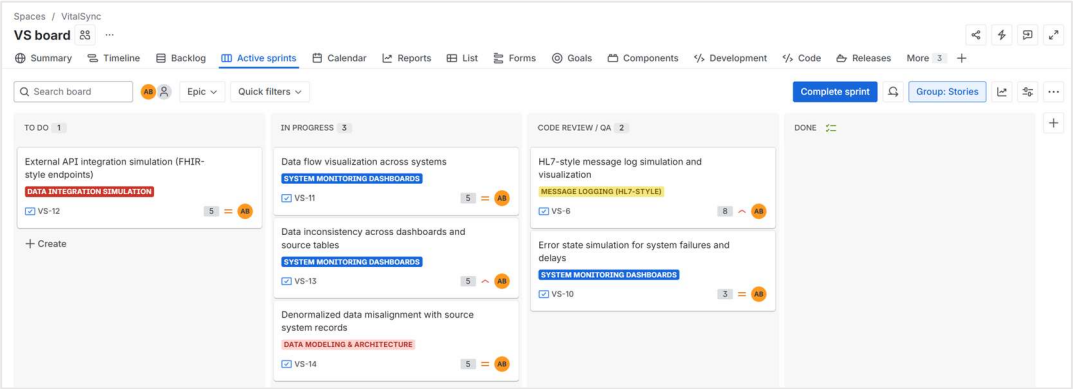
1	**Data Consistency Across Simulated Systems**
	• Ensuring logs, system status, and dashboards stayed aligned
	• Avoiding mismatches between:
	  ○ backend data
	  ○ UI representation
		• Centralized data updates
		• Ensured single source of truth in database

2	**Cross-System Data Flow Simulation**
	• Simulating realistic message flows between systems
	• Representing:
	  ○ successful transactions
	  ○ failures
	  ○ delays
		• Designed structured message logs
		• Implemented status states (success, error, pending)

3	**Observability Design** • Deciding what "visibility" means in a simulated system	Focused on: • system health • message flow • integration status
4	**Dashboard Synchronization** Ensuring dashboards reflected real-time-like system behavior	Structured update logic between backend and UI

Table 3 – Sprint 2 Key Challenges: System-level challenges encountered while simulating real-world data pipelines and observability across distributed systems. These challenges reflect common failure points in distributed data systems, including data inconsistency, unreliable pipelines, and lack of visibility.

4.3.2 Sprint 2 In-Progress Screenshot



Screenshot 6 - Sprint 2 In-Progress State: Sprint 2 in progress, highlighting development of message logging, integration tracking, and data flow visualization. Key efforts focused on resolving data consistency challenges across dashboards and underlying data structures while ensuring realistic system behavior and synchronization.

4.4 Sprint Completion

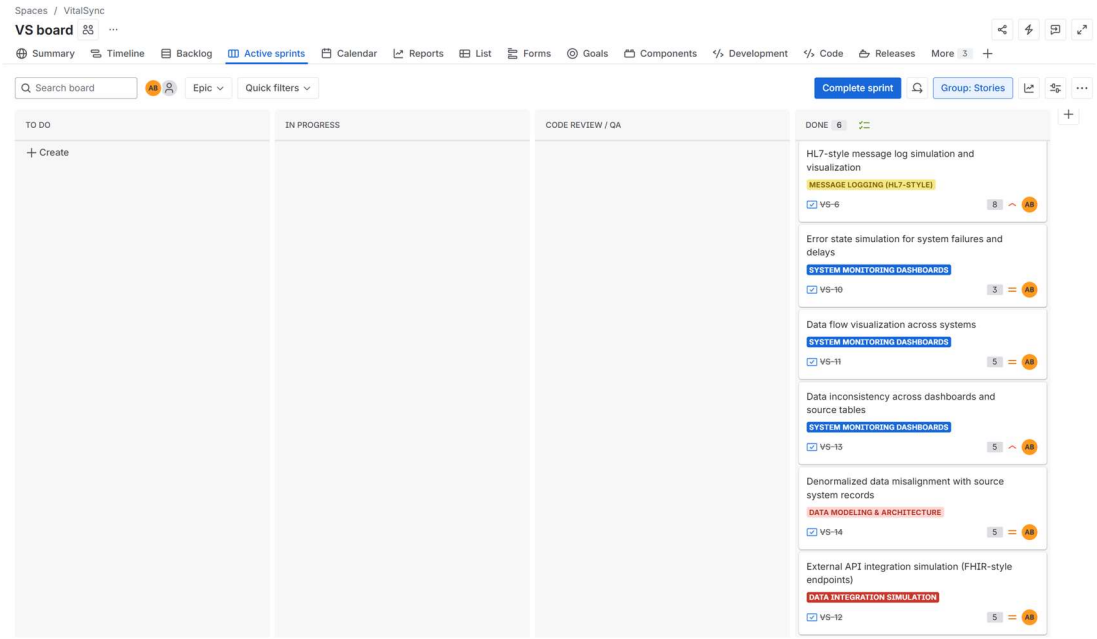
4.4.1 Outcome:

Delivered:

- HL7-style message logs and event simulation
- integration monitoring
- system health dashboards
- simulated multi-system architecture
- Enabled cross-layer visibility across clinical, integration, reporting, and research domains

Data consistency and modeling issues identified during integration were resolved within the sprint, reflecting iterative system refinement and alignment across platform components.

4.4.2 Sprint 2 Completion Screenshot



Screenshot 7 - Sprint 2 Completion: Sprint 2 completed with delivery of HL7-style message logging and event simulation, integration monitoring, and enhanced dashboard capabilities. Data consistency issues were resolved, resulting in aligned system behavior across logs, dashboards, and underlying data models.

5. Metrics & Outcome

5.1 Product Outcomes

- Enabled visibility into system health, data flows, and integration status
- Simulated realistic multi-system interactions for monitoring scenarios
- Established structured data model supporting observability and logging

5.2 Delivery Outcomes

- Delivered MVP across structured iterations with controlled scope
- Built end-to-end system from data modeling to visualization
- Improved system clarity through unified dashboards and logs
- Defined a scalable observability model for multi-system monitoring environments
- Defined KPI-based monitoring thresholds to simulate real-world system health evaluation, including success rates, latency, and failure counts
- Metrics were categorized into healthy, warning, and critical states to support rapid identification of integration issues and system degradation

6. Project Management Insights

- **System Thinking:** Focused on designing the system as interconnected components and not isolated features.

- **Prioritization:** Delivered architecture first, observability second, and enhancements last
- **Dependency Management:** Managed dependencies between data model, backend logic, and dashboard UI.
- **Problem Solving:** Resolved complex challenges in data consistency, system synchronization, and observability design.
- **Execution:** Maintained structured delivery with iterative builds, continuous testing, and controlled scope.

7. Supporting Artifacts

- Architecture diagrams
- Database schema
- Dashboard screenshots
- Message log examples
- Jira Backlog
- Sprint Boards (Sprint 1 & Sprint 2)

8. Security and Data Considerations

8.1 Context

Although VitalSync is a simulated platform, it models healthcare-style data environments where:

- sensitive information (patient/system data) is typically present
- regulatory considerations (e.g., GDPR, HIPAA-like principles) are critical
- data integrity and system trust are essential

The platform incorporates governance principles typical of healthcare interoperability systems, including traceability and controlled data exposure.

8.2 Problem

Designing a system that simulates real-world integrations required balancing:

- realistic data flows and logs
- without exposing or relying on actual sensitive data
- while maintaining credibility of the system design

8.3 Approach/Design Decisions

No.	Approach / Decision	Impact

1	**Data Anonymization & Simulation**
	• Used **synthetic / simulated datasets only**
	• Avoided:
	  ○ real patient data
	  ○ identifiable information
	• Structured logs to reflect:
	  ○ system events
	  ○ message flows
	  ○ statuses
		Ensured realistic behavior **without privacy risk**

2	**Separation of Concerns (Data vs Presentation)**
	• Maintained clear separation between:
	  ○ backend data structures
	  ○ frontend display logic
		Reduced risk of:
		• Unintended data exposure
		• Inconsistent data handling

3	**Data Integrity & Consistency Controls** • Centralized data updates to ensure: ○ single source of truth	Prevented: • Mismatched logs • Inconsistent system state Critical for: • observability accuracy • system trust

4	**Controlled Input & Validation**
	• Implemented validation on:
	  ○ data inputs
	  ○ simulated system events
		Prevented:
		• invalid states
		• unrealistic system behavior

5	**Observability Without Overexposure**
	• Designed dashboards to show:
	  ○ system status
	  ○ message flow
	  ○ integration health
		Avoided displaying:
		• unnecessary raw data
		• sensitive-level detail

Table 4 – Security and Data Considerations Approach: Design decisions ensuring data privacy, integrity, and system reliability within a simulated healthcare environment, balancing realistic system behavior with safe data practices.

9 Product Thinking: What I'd Do Next

To evolve VitalSync from a prototype into a production-ready platform, I would focus on four areas:

1. **Real Integrations & Data Pipelines**
   - Replace simulated data flows with real-time integrations (FHIR APIs, streaming pipelines)
   - Introduce event-driven architecture for scalable ingestion and processing
   - Enable near real-time observability across distributed systems
2. **Advanced Observability & Alerting**
   - Implement alerting and anomaly detection on system health and message flows
   - Introduce SLA monitoring (latency thresholds, failure rates)
   - Add root-cause analysis capabilities across system layers
3. **Platform & User Expansion**
   - Introduce role-based access (clinical, operations, leadership views)
   - Enable multi-tenant architecture for broader platform usage
   - Expand reporting with configurable dashboards and export capabilities
4. **Data Governance & Compliance**
   - Extend governance model to support audit trails, access controls, and compliance reporting
   - Align platform design with real regulatory frameworks (GDPR, HIPAA)

10 Future Improvements

Potential enhancements may include:

- Expanded dashboard analytics
- Enhanced API integration examples
- Authentication and user roles
- Additional interoperability scenarios